LECTURE 4

MLflow for GenAI

Tracing, evaluation and observability

4 November 2026

LLMOps with Databricks



LECTURE 4

Agenda

01 **MLflow components**
Tracking, registry and tracing

02 **Storage and governance**
Regular ML, GenAI and Unity Catalog

03 **Traces and spans**
Inspect and instrument agent execution

04 **Evaluation**
Test behavior, define scorers and inspect failures

05 **Tracking and registration**
Prepare an agent for Model Serving



MLflow components



MLflow components

01

Experiment tracking

Main use: machine learning. Compare training runs, parameters, metrics and artifacts.

Also used to log agent artifacts for Model Serving.

02

Model registry

Manage model versions and aliases in Unity Catalog. Share models across workspaces.

ML models and agents served as models.

03

MLflow tracing

Understand what happens inside an agent. OpenTelemetry-compatible and closely tied to evaluation.

Introduced in MLflow 2.14.0 · June 2024

For an agent behind a Model Serving endpoint, also log the agent and register a model version.



LLM observability tools also support evaluation

Tool / ecosystem	Observability	Evaluation
Pydantic Logfire + pydantic-evals	✓	✓
MLflow	✓	✓
Arize Phoenix	✓	✓
Opik (Comet)	✓	✓
Langfuse	✓	✓

Human review and feedback are also supported by MLflow, Phoenix, Opik and Langfuse.

> **Observability and evaluation belong in the same workflow.**



Model flavors and payloads

Classic ML

sklearn · statsmodels
pytorch · catboost
spacy · lightgbm
xgboost · tensorflow

Custom pyfunc.PythonModel

REQUEST ALTERNATIVES

```
{"dataframe_split": {
  "columns": ["x"], "data": [[1]]}}
{"dataframe_records": [{"x": 1}]}

{"instances": [1.0, 2.0, 5.0]}
{"inputs": [["cheese"], ["Crackers"]]}
```

instances / inputs: tensor models

```
{"predictions": [[120.5]]}
```

AI agents

dspy
langchain
llama_index

Custom pyfunc.ResponsesAgent

RESPONSES-STYLE INPUT

```
{"input": [{
  "role": "user",
  "content": "Find new papers"
}]}
```

ResponsesAgent defines the request and response schema.

Other agent flavors: check the model signature; chat format is not universal.

Foundation models

transformers

Deploy supported model families and architectures

Interface **Depends on the task**

SUPPORTED ENDPOINT TASKS

Chat · Embeddings · Completions

```
{"messages": [{
  "role": "user", "content": "Hi"
}]}

{"input": "Text to embed"}

{"prompt": "Once upon a time"}
```



Storage and governance

Regular ML and GenAI experiments

REGULAR ML

Training experiments

Workspace

Experiments with training runs,
parameters and metrics

Unity Catalog

Registered ML models:
versions and aliases

GENAI

Tracing and evaluation

Experiment

Organizes agent traces
and evaluation results

Unity Catalog

Trace data can be stored in UC
with governed access

Model Serving is the overlap: log the agent in an MLflow run, then register it in UC.



Why GenAI changes the data sensitivity

TRADITIONAL ML

Often aggregate information

learning_rate	0.01
max_depth	8
validation_f1	0.89

Illustrative run metadata

GENAI

The content of a request

“Summarise this confidential customer case…”

Retrieved document excerpts

Tool arguments and the full response

A trace can become another copy of protected business data.

> Traditional ML artifacts and logged samples can also contain sensitive data.



Workspace administration and trace access

Workspace-managed traces

Workspace admin → Experiment ACLs → **Trace contents**

Workspace admins can manage experiment permissions and potentially read sensitive payloads.

UC-backed traces

Application team → UC privileges → **Trace tables**

Govern trace access through catalog, schema and table permissions.

Ownership matters. Workspace admins may also own a catalog or hold UC privileges.



Registered models in Unity Catalog

Workspace A

Develop and compare

Log and evaluate an agent version

↔

UNITY CATALOG

catalog.schema.arxiv_agent

Version 1 | Version 2 | Version 3

Governed model name and aliases

↔

Workspace B

Load and deploy

Use an authorised model version

Same metastore, appropriate privileges, and permitted workspace–catalog bindings.



Traces and spans



Anatomy of a trace

Two key components of every trace

Trace Info

- experiment_id
- duration
- status
- tags

Trace Data

- LLM span
- Tool span
- Retrieval span

Trace info

Experiment ID and trace status

Total execution latency

Tags, session ID and custom metadata

Trace data

Instrumented span objects

LLM invocations and tool calls

Step-by-step execution payload

Span types for GenAI operations

Categorize the operation performed by each span

LLM	Language model inference
TOOL	External tools, functions or APIs
RETRIEVER	Vector search and context retrieval
MEMORY	Conversational memory reads and writes
EMBEDDING	Generating vector embeddings
RERANKER	Re-ranking retrieved results



Using the tracing APIs

Choose the API for the operation you need

`@mlflow.trace()`	Instrument a function with a decorator
`mlflow.update_current_trace()`	Add custom metadata to the active trace
`mlflow.start_span()`	Control the scope of a span manually
`mlflow.search_traces()`	Search and filter recorded traces



Code example: basic tracing

Decorator

```
import mlflow

mlflow.set_experiment(
    cfg.experiment_name
)

@mlflow.trace
def add_numbers(x: int, y: int):
    return x + y

add_numbers(2, 3)
```

Inputs and return value are captured automatically.

Manual span

```
def add_manually(x: int, y: int):
    with mlflow.start_span(
        name="add_numbers"
    ) as span:
        span.set_inputs({"x": x, "y": y})
        result = x + y
        span.set_outputs(result)
        return result

add_manually(2, 3)
```

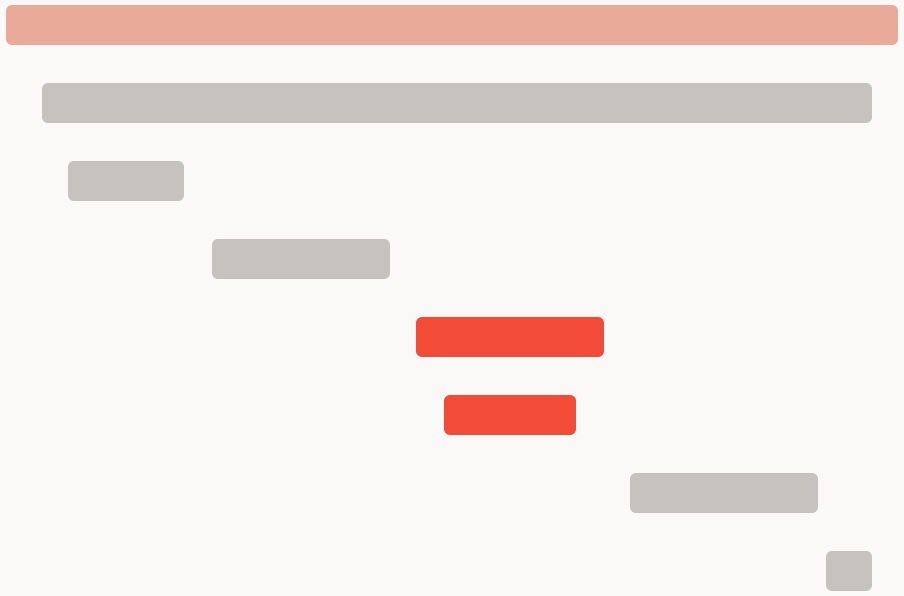
Set inputs and outputs explicitly within the span.



One request through the arXiv agent

One request, with nested operations you can inspect.

predict_stream	AGENT
call_and_run_tools	CHAIN
memory_load	RETRIEVER
LLM call	LLM
execute_tool	TOOL
search_papers	retrieval work
LLM call	LLM
memory_save	CHAIN



Illustrative hierarchy and timing. The actual spans depend on the executed path and integrations.

What a trace helps us investigate

01

Slow response

Which span dominates the request? Is the agent making repeated tool calls?

02

Poor answer

What did retrieval return? What did the model receive? Did memory load the expected history?

03

Unexpected cost

How many model calls ran? Where available, compare token usage by span and version.

Execution succeeded. That alone does not tell us whether the answer was useful.



UC trace setup

```
from mlflow.entities.trace_location import UnityCatalog

experiment = mlflow.set_experiment(
    experiment_name=cfg.experiment_name,
    trace_location=UnityCatalog(
        catalog_name=cfg.catalog,
        schema_name=cfg.schema,
        table_prefix="llmops",
    ),
)
```

The destination is configuration

The experiment is the entry point.

Catalog, schema and prefix identify the trace tables.

The full notebook checks the binding and grants access.



UC traces: four tables and two views

Catalog Explorer → arxiv → Tables

arxiv

Trace objects · prefix: `llmops`

TABLE	`llmops_otel_annotations`
TABLE	`llmops_otel_logs`
TABLE	`llmops_otel_metrics`
TABLE	`llmops_otel_spans`
VIEW	`llmops_trace_metadata`
VIEW	`llmops_trace_unified`

4 tables

OpenTelemetry annotations, logs, metrics and spans

2 views

Trace metadata and a unified representation for querying

Use later for monitoring

Query with SQL, aggregate trends and build monitoring dashboards.

Permissions and deployment checks

Identity	Access to check
Human reader	Intended trace data and SQL warehouse access
Agent runtime identity (e.g. SPN)	USE CATALOG, USE SCHEMA and trace-object permissions
Unrelated user	No unintended access through inherited grants

WRITE

Write a trace as the agent's runtime identity

READ

Inspect it as the intended reader

To write and read traces, the agent's runtime identity needs **SELECT / MODIFY** on trace tables, **SELECT** on views, plus **USE CATALOG** and **USE SCHEMA**.



Searching UC traces uses a SQL warehouse

```
import os
import mlflow

os.environ["MLFLOW_TRACING_SQL_WAREHOUSE_ID"] = (
    cfg.warehouse_id
)

traces = mlflow.search_traces(
    experiment_ids=[experiment.experiment_id],
    max_results=20,
)
```

Why a warehouse?

MLflow queries the trace data stored in Unity Catalog.

Before searching

Set the warehouse ID in the environment variable.

The caller needs warehouse **CAN USE** and UC read access.



A parent span and two child spans

```
@mlflow.trace(span_type=SpanType.TOOL)
def search_database(query):
    return ["Example paper"]

@mlflow.trace(span_type=SpanType.LLM)
def call_llm(prompt):
    return "Simulated answer"

@mlflow.trace(span_type=SpanType.CHAIN)  # PARENT
def process_query(query):
    results = search_database(query)  # CHILD 1
    return call_llm(f"{query}: {results}")  # CHILD 2

process_query("Find papers on agents")
```

Parent: process_query

Calling it starts the span created by its decorator.

process_query · parent

- search_database · child
- call_llm · child

Both decorated calls run inside the active parent span.

Illustrative functions with simulated results.

Metadata connects requests to versions

```
mlflow.update_current_trace(
    metadata={
        "mlflow.trace.session": session_id,
        "environment": env,
    },
    tags={"git_sha": git_sha, "model_version": version},
    client_request_id=request_id,
)
```

REQUEST CONTEXT

Three useful links

Request ID
One client request

Session ID
Related conversation turns

Version
The implementation that ran



OUR COURSE AGENT

Our agent: arXiv papers → LinkedIn posts

On request, use arXiv papers to write a LinkedIn post about the user's topic.

SCHEDULED DATA PIPELINE

arXiv papers
PDF content + metadata

→

Prepare data
Parse documents
Store Delta tables

→

Make it searchable
Vector search / pgvector
Genie for paper metadata

↓ Paper retrieval and metadata tools

User request
"Write a LinkedIn post about agent memory"

→

arXiv agent
LLM via AI Gateway
Tools + Lakebase memory

→

LinkedIn draft
A post grounded in the retrieved papers

Application: Databricks Apps / Model Serving

MLflow: trace execution → evaluate the output



Evaluation

Observability and evaluation work together

Observability

What happened, and why?

Inputs, outputs, retrieval and tool calls

Latency, token usage and cost

Evaluation

How well did it perform?

Correctness, relevance and safety

Code checks, human review and LLM judges

Traces provide context for scoring



Low scores guide investigation

Traces explain the execution. Evaluations assess its quality.

EVALUATION

How do we know the output is correct?

Dimension	Traditional ML	LLM-based applications
What “good” means	Defined target and measurable prediction error	Task-specific criteria; several valid answers
Quality signals	Precision, recall, F1; RMSE, MAE, R^2	Correctness, relevance, groundedness, task success
Evaluation methods	Compare predictions with known outcomes	Code checks, human review and LLM judges
Ground truth	Labels may arrive hours, days or weeks later	References may be delayed, costly or unavailable
Early signals	Estimated performance; input and prediction drift	Judge scores, user feedback, shifts in requests
Diagnosing failures	Slice analysis, feature drift, SHAP attributions	Trace prompts, retrieval, tool calls and responses



Evaluation data starts with expected behavior

ONE ARXIV EVALUATION CASE

“Write a LinkedIn post about agent memory”

Context

The requested topic and the retrieved arXiv papers

Expected behavior

Address the topic, cite real papers, preserve their claims and avoid invented findings.

Build a reviewed dataset

- Normal requests and meaningful edge cases
- Prior turns or evidence where needed
- Explicit expectations
- Regression cases from real failures

EVALUATION

Easy checks can miss a real problem

A passing score does not guarantee a useful response.

The exchange

User

“Same issue as last month”

System

“Can you describe the problem?”

The evaluation

Check	Method	Result
Politeness	LLM-based	✓ Pass
Relevance	LLM-based	✓ Pass
≤ 300 characters	Code-based	✓ Pass

THE MISSING CHECK

Did the response use the **previous conversation?**



Getting started with evals



1. Create examples
Start with 10–20 carefully reviewed cases.

2. Run test cases
Process them through the current agent.

3. Evaluate with the business
Did the output meet expectations?

4. Identify error patterns
Explain why an output failed.

5. Decide on key metrics
Define what you actually need to measure.

6. Iterate and expand
Refine prompts or tools and repeat.

EVALUATION

Metrics define what; scorers define how

Metrics define what to measure. Scorers define how.

Code-based scorers

Best for

Format, length, tool calls

Strength

Fast, low-cost and repeatable

Limitation

Limited nuance and context

Model-based scorers

Best for

Relevance and contextual quality

Strength

Flexible for open-ended outputs

Limitation

Variable scores: calibrate with humans

Human scorers

Best for

Business expectations and edge cases

Strength

Expert judgment and calibration

Limitation

Slow and costly to scale



EVALUATION

MLflow LLM scorers: simplicity and control

Predefined scorers

Simplicity **Quickest setup**

Control **Preset criteria**

Built-in classes
↓
Correctness · RetrievalGroundedness
↓
MLflow constructs the judge prompt

Select a built-in scorer; MLflow supplies its evaluation template.

Best for

- Quick baseline evaluations
- Standard quality criteria
- A few lines of code

Guidelines-based scorers

Simplicity **Simple rules**

Control **Your criteria**

“Response must be polite”
↓
LLM checks the rule
↓
Pass / fail + rationale

Write plain-language rules and evaluate whether each response complies.

Best for

- Style and business rules
- Required or excluded information
- Shared pass / fail criteria

Template-based scorers

Simplicity **More setup**

Control **Custom rubric**

{{ inputs }} · {{ outputs }}
↓
Custom grading instructions
↓
Your score type + rationale

Use make_judge() for a custom rubric. Optionally align it with human feedback.

Best for

- Domain-specific judgments
- Nuanced scoring scales
- Versioning and alignment



EVALUATION

MLflow scorers: choosing the evidence

Scorer	What does it evaluate?	Evidence
RelevanceToQuery	Does it address the request?	Input and response
Correctness	Does it support expected facts?	Reference expectations
RetrievalGroundedness	Does retrieved evidence support it?	Retrieval spans
Guidelines	Does it follow our criteria?	Inputs, outputs and rules

Full MLflow scorer reference →

> Choose a scorer that can see the evidence needed for the judgment.



EVALUATION

Guidelines and custom judges

GUIDELINES

Pass / fail

“When earlier attempts failed, acknowledge them and offer a new approach.”

Shared natural-language rules across examples.

CUSTOM JUDGE

A task-specific rubric

1. Repeats a failed suggestion
2. Acknowledges, but offers no useful next step
3. Acknowledges and gives a relevant next step

Custom instructions, template inputs and output type.

> Both approaches need calibration against human feedback.



A guideline for conversation context

```
from mlflow.genai.scorers import Guidelines

context_guideline = Guidelines(
    name="acknowledges_previous_attempts",
    guidelines=[
        "If the user says earlier attempts failed, acknowledge "
        "those attempts and offer a new approach or escalation."
    ],
    model=JUDGE_MODEL,
)
```

GUIDELINES

A conditional rule

Only apply the expectation when the user describes failed attempts.

The judge needs the relevant conversation context.



EVALUATION

A custom judge with an explicit rubric

```
from mlflow.genai.judges import make_judge

context_judge = make_judge(
    name="context_handling",
    instructions=(
        "Given {{ inputs }} and {{ outputs }}, score 1–3: "
        "1 = repeats a failed suggestion; "
        "2 = acknowledges failure but gives no useful next step; "
        "3 = acknowledges failure and offers a relevant next step."
    ),
    feedback_value_type=int, model=JUDGE_MODEL,
)
```

CUSTOM JUDGE

A defined scale

1 Repeats a failed suggestion

2 Acknowledges, but does not help

3 Offers a useful next step



EVALUATION

Our course scorers: useful but incomplete

Our current checks

- ✓ Under 350 words
- ✓ Polite tone
- ✓ Engaging opening

? Does it represent the paper correctly?

A short, polite post can still misstate the research.

> Add factual support and task-specific expectations. Keyword matching alone cannot establish correctness.



EVALUATION

Judge reliability needs human calibration

A shared rubric

Give the judge and reviewers the same criteria and context.

Human feedback

Judge feedback

Inspect disagreements

False passes

False failures

Ambiguous rules

Missing evidence

Check fresh examples

Reassess on cases that did not shape the judge.

LLM judges provide scalable feedback. Their scores still need scrutiny.

EVALUATION

Calibration to one person is not enough

A COMMON PITFALL

Optimize for agreement with one human

The judge can learn that person's preferences, blind spots and inconsistent labels.

High agreement alone does not establish that it can replace a reviewer.

THE DECISION WE NEED TO MAKE

Is it good enough to replace a human?

Compare the judge with multiple qualified reviewers using the same rubric and context.

Test on fresh examples and account for the cost of mistakes.

> Define replacement for a **specific task and review decision**, not for human judgment in general.



EVALUATION

Is an LLM judge good enough to replace a human?

Alt-Test

1. Leave one human annotator out.
2. Compare the LLM and that human with the same remaining annotators.
3. Repeat for every human.
4. Apply the cost-benefit adjustment and false discovery rate (FDR) correction.

Winning rate ≥ 0.5

Supports replacement for the tested task and settings, after the statistical tests.

Calibration sample: 3+ humans; the paper discusses 50–100 items. Sample needs vary.

Match instructions

Give humans and the LLM the same written rubric and context.

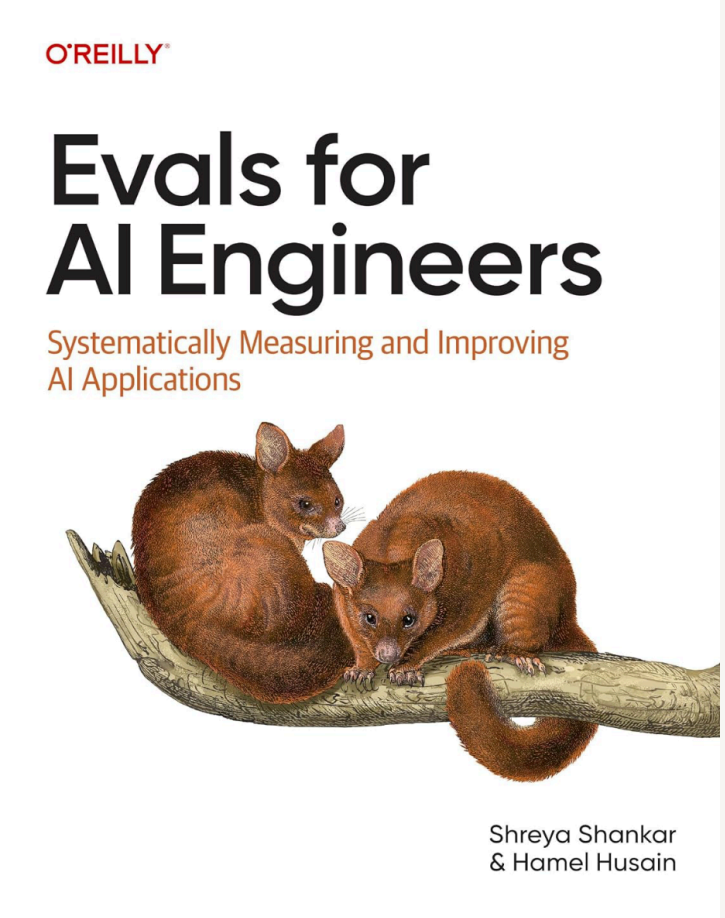
Clarify class definitions

Inspect confused class pairs. Refine ambiguous definitions before blaming the model.

FDR correction controls false “wins” across multiple human comparisons.

Calderon, Reichart & Dror · ACL 2025 · Alt-Test paper

Recommended resource on evals



RECOMMENDED READING

Evals for AI Engineers

Systematically Measuring and Improving AI Applications

Shreya Shankar & Hamel Husain
O'Reilly

Continue with a systematic approach to test cases, error analysis and evaluation workflows.

Open the book on O'Reilly →

Tracking and registration

Tracking and registration complete the workflow

01

Evaluate

Inspect individual cases and decide whether the version is ready.

→

02

Track & log

Record metrics and log the configured agent artifact in an MLflow run.

→

03

Register

Create a UC model version and update its alias deliberately.

> Recording evaluation metrics does not automatically enforce a release gate.



The agent artifact for next week

```
with mlflow.start_run(run_name="arxiv-agent"):
    model_info = mlflow.pyfunc.log_model(
        name="agent", python_model=agent,
        input_example=test_request, model_config=model_config,
    )
    mlflow.log_metrics(results.metrics)

registered_model = mlflow.register_model(
    model_uri=model_info.model_uri,
    name=f"{cfg.catalog}.{cfg.schema}.arxiv_agent",
)
```

NOTEBOOK 4.4

Ready for the next step

The run records metrics and the agent artifact.

The registry gives that artifact a model version.

Registration does not deploy an endpoint.



Homework



HOMEWORK

Homework: trace, evaluate and explain

01 Instrument your agent

Configure its trace destination. Show a request with meaningful spans and version/session metadata.

02 Evaluate realistic behavior

Create a small reviewed dataset. Add a code check and a domain-specific judge with clear expectations.

03 Investigate a failure

Attach a sanitized trace example, explain the failure, and add a regression case.

04 Submit your work

Open a pull request with the implementation, evaluation findings and intended trace permissions.



Questions?

